



Training a coding agent through a harness you did not write

Sergio Paniego | [@sergiopaniego](#) 🤖 Hugging Face

Kernel Panic, AI Open Models Conference
Madrid, 06 Oct 2026



THE SAME WEIGHTS, ON THE SAME TASKS

62%

VS

33%

Mini-SWE-Agent

Claude Code

The only difference is the program wrapped around the model.





An agent is a model plus a harness

Claude Code

Codex

OpenCode

Cursor

Cline

Pi

Goose

Mini-SWE-Agent

runs the loop

it calls the model again and again, and decides when to stop

picks the tools

the model only gets the tools this harness exposes, under its names

writes the context

what the model reads is assembled, trimmed and compacted by the harness

reads the reply

and rejects it if the format is not the one it expects

Change the harness, and you change what the model sees and may do.

None of them was written with training in mind, and we do not change a line of their code.



Every harness speaks its own dialect

Harness	Anthropic Messages	OpenAI Responses	Google generateContent	chat completions	plain text	Tool calls
Claude Code	●					yes
Codex		●				yes
OpenCode	●	●	●	●		yes
Goose	●		●	●		yes
SWE-agent				●		yes
Mini-SWE-Agent		○		●	○	one tool
OpenHands SDK				●		yes
Aider				●		no
Qwen Code			●	○		yes
Kimi CLI				●		yes
Terminus 2				●		no
Hermes		three APIs, switchable				yes

LEGEND

● speaks it by default ○ also supports it





Leaving the harness you trained in can end the run

Orchard took finished coding models and ran each one under three harnesses, on the same tasks. One of the three, Kimi-CLI, none of them had used in training. Two things went wrong.

degraded

the model still works, and solves less

OpenSWE-32B keeps 54.9% when it moves to Mini-SWE-Agent, then falls to 3.6% under Kimi-CLI, from 62.4% at home. On Terminal-Bench 2.0 it scores zero.

catastrophic

the output stops being usable at all

Scale-SWE stops producing valid tool calls anywhere but the harness it was trained in. Orchard has no number to report for those cells.

Kimi-CLI was new to both of them, and they had each seen two harnesses in training.



Three ways a model overfits to its harness

Format

overfitting

anchored to one action
format

trained in a tool-calling harness

```
"tool_calls": [{"name": "edit_file", ...}]
```

WHAT YOU SEE

The JSON arrives as chat text. Nothing is parsed, nothing is edited, and the turn is scored as a non-answer.

dropped into aider, which takes edits in prose

```
<<<<<<< SEARCH ... ===== ... >>>>>>> REPLACE
```

Context structure

overfitting

anchored to one way of
stacking history

trained where each result is its own message

```
role: "tool" → one message per result
```

WHAT YOU SEE

Its own earlier results are no longer where it expects them, so it repeats work it has already done.

dropped into a harness that compacts history

```
role: "user" → one summarised turn
```

Control flow

overfitting

anchored to someone else's
retry and stop rules

trained where the harness retries for it

```
exit 1 → harness retries, then stops at step N
```

WHAT YOU SEE

It runs the same failing command again, because deciding when to stop was never its job.

dropped into a harness that just reports and waits

```
exit 1 → error returned to the model
```

The three modes are from the KAT-Coder report. One fix covers all three: vary the harness during training.



This is how frontier labs train coding models today

In 2025 most model reports did not say which harness a number came from. These are all 2026, and every one trains inside real harnesses on purpose, over large pools of RL environments at once.

Kimi K3	builds Claude Code, Codex and others from composable modules, and trains across them
Qwen3-Coder-Next	generates its agentic coding data in six different harnesses
MiMo-V2.6	trains on a pool of task-adapted mini-harnesses
Poolside Laguna	1.3B tokens of OpenHands, OpenCode and Mini-SWE-Agent trajectories
Liquid LFM2.5	picks a harness at random for each training task

OpenForgeRL checked whether the variety costs peak performance. It does not: **three** harnesses beat one, **even on that one's home turf** (48.5 vs 46.0).



Open weights, closed training surface

WHAT YOU GET

- + Open weights, often a strong model
- + A benchmark table
- + A paragraph naming the harnesses

WHAT YOU DO NOT

- The harness pool, wired the way they ran it
- The environment mix, and how it was sampled
- The capture layer that turns a harness loop into training samples
- The trainer wiring on the other side of it

Every one of those reports describes the practice without handing you **anything you can run.**



SO WHAT IS MISSING

An **open** way to train inside **any** harness, without touching its code

It has to capture everything RL needs from a program you do not control, and feed whichever trainer you already use.

AND IT DOES NOT TAKE A FRONTIER BUDGET

41.8% → **56.4%** on SWE-bench Verified

Agent Lightning, Qwen3.5-9B, with roughly 6,000 training examples



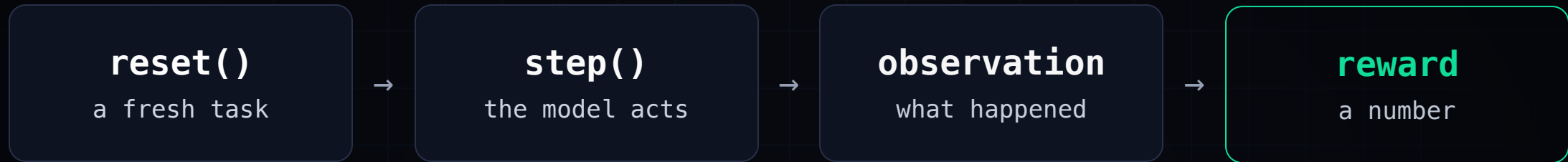
Agentic RL, in four steps

- 1 Give the model a task, and let it try the same one several times
- 2 Grade each attempt with a number. Did the tests pass?
- 3 Compare the attempts against their own average
- 4 Make the better-than-average tokens more likely, the worse ones less

That is **GRPO**. The group is its own baseline, so you never need a second value model. **Agentic** just means each attempt is a whole session with tools, with a filesystem and a shell behind it.



An RL environment is where it practises



A coding harness already has every one of those.



state ← the filesystem the agent works in

actions ← its own write, bash and edit tools

Normally the trainer drives. Here the harness does

White box

Black box



*“In traditional agentic RL, the training engine owns the environment interaction loop. In **harnessed agentic RL**, the harness owns this loop, while the training engine observes only a sequence of LLM request–response pairs.”*

Agent Lightning, Microsoft 2026



OpenEnv is the seam

What it is

a standard interface for RL environments: Gymnasium-style reset, step and state, over a client and server transport, with typed actions and observations

How it ships

packaged as Docker, publishable to the Hub, so an environment is a repo you can pull

Who steers it

started as a Meta PyTorch and Hugging Face collaboration, now a committee of twelve organisations, BSD-3

It does not train anything and it does not define rewards. It is what the harness, the environment and the trainer all plug into, and it is where the **capture proxy** lives.



TRL is the trainer

Hugging Face's post-training library. One trainer class per method, same shape for all of them.

SFTTrainer

DP0Trainer

GRPOTrainer

RL00Trainer

KT0Trainer

RewardTrainer

DistillationTrainer

async GRPO

Generation and training run apart. One GPU trains while a second serves the policy with vLLM and keeps producing rollouts, so training never waits for the slowest agent. The price is that rollouts come from weights two optimizer steps old, never more than four.



Harbor brings the tasks and the sandboxes

40+

harness adapters

10

validated end to end in OpenEnv

1

fresh sandbox per rollout

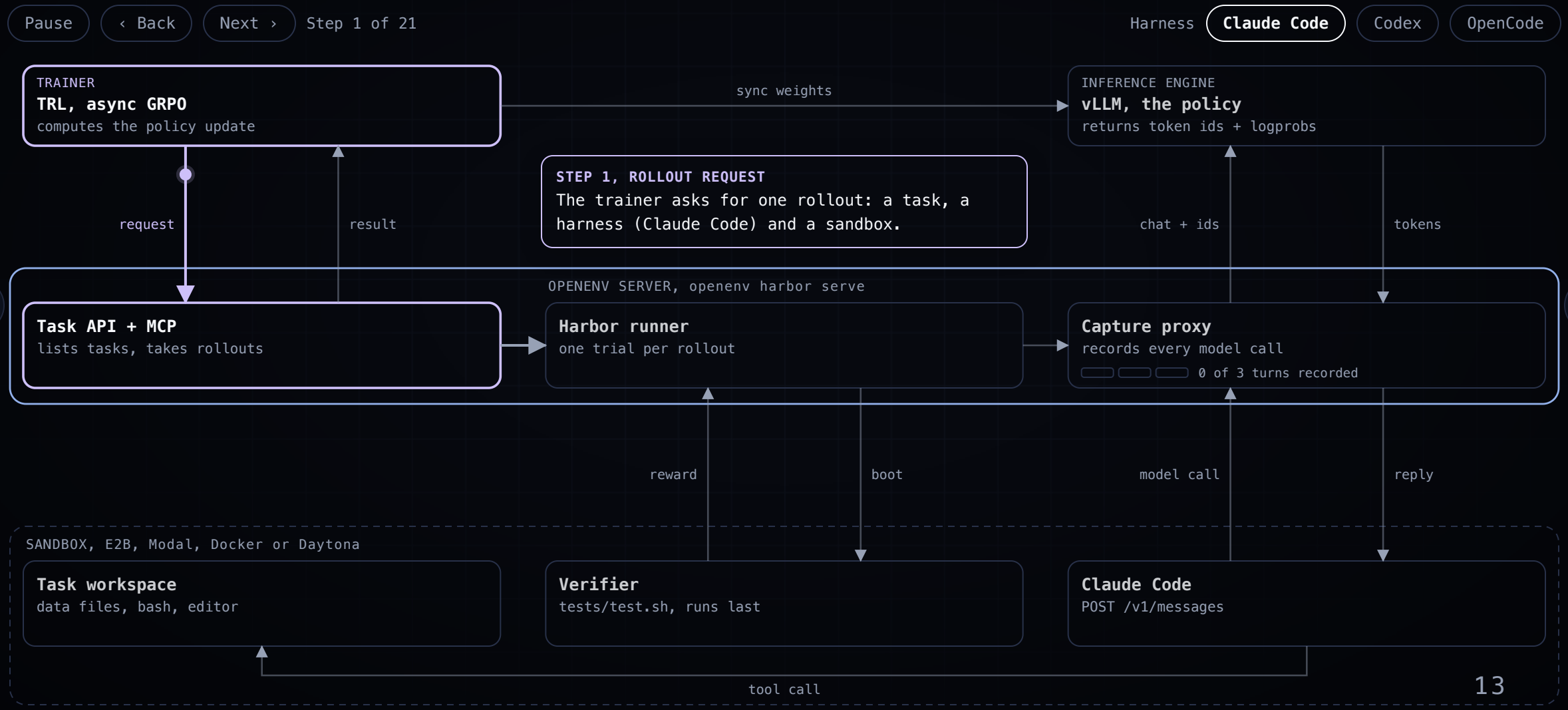


A harness needs a task and a box to run it in. Wire them yourself and you write the glue for every harness against every sandbox backend, with the harness sewn into your training script. Harbor keeps the task, the harness and the sandbox independent of each other, so each one becomes something you pick per rollout.

```
env = HarborEnv(task=..., harness="claude-code")
```



One rollout, end to end





The capture proxy

Pause

1/11 the agent is pointed at the proxy, and its API key is the capture session id



SANDBOX
the agent

key = sess_7f3a
unmodified

CAPTURE PROXY
the only new part

- detect dialect
- normalise, ids on
- record

INFERENCE ENGINE
the policy

ids + logprobs on
tokenises to serve



CAPTURE STORE ☒ prompt, masked ☐ sampled, one logprob each

0 trainable tokens, turn 1

turn 1 waiting for the first completion

The proxy probes the engine before it trusts it. An engine that will not return token ids and logprobs still serves the rollout, and the trace comes back marked evaluation only. Every hosted API lands there.



In code

```
env = HarborEnv.from_hub(  
    "FineEnvs/smoldataenv-multi-harness-harbor",  
    harness=random.choice(HARNESSES),  
)  
  
worker = HarnessRolloutWorker(  
    harness_session_factory=env.session_factory,  
    harness_adapter=None,  
    rollout_reward_fn=reward,  
)  
  
AsyncGRPOTrainer(model=..., rollout_worker=worker).train()
```

tasks and sandboxes
picked per rollout
the agent owns its loop
correctness and tool calls

Swapping the harness is **one argument**.



SO WE RAN IT

One 2.6B model, **four**
harnesses, 1,000 steps

OpenCode

Claude Code

Codex

Mini-SWE-Agent





The tasks: **SmolDataEnvs**

One task is a real table, a question about it, and a gold answer. They come from real data-science notebooks, through the jupyter-agent dataset.

5,000

training tasks

250

held-out, harder on purpose

471

Kaggle datasets behind them

Verified by a checker, not judged by a model. Every task had to be solved in a live sandbox before it was kept, and grading is an exact comparison against the known answer. No model sits in the reward path, so the signal cannot drift.

SmolDataEnvs

the tasks as plain rows, load it and prompt any model

-harbor-train

the same tasks as runnable, sandboxed Harbor environments

-multiharness-sft

agent trajectories to fine-tune on, per harness



The setup

model	LFM2.5–2.6B, trained by Liquid AI in Hermes Agent and OpenClaw, neither of them ours
tasks	1,000 of those training tasks, 400 medium and 600 hard, in one fixed order
test	250 held-out tasks, each run under all four harnesses: 1,000 cells
two runs	one in OpenCode alone, one rotating the four across GRPO groups
recipe	async GRPO in TRL, 1,000 steps, 2x H100, one E2B sandbox per rollout
time	32 h for OpenCode-only, 46 h for multi-harness

[LiquidAI/LFM2.5–2.6B](#) [FineEnvs/SmolDataEnvs](#)

Both runs follow the same task order, and both are evaluated the same way, including under the three harnesses the OpenCode-only run never saw.



What we rewarded

correctness

1 for a correct answer, 0 for a wrong one.
Graded by the task's own verifier.

tool efficiency

at most 0.1, and only on a correct answer, for
taking fewer calls.

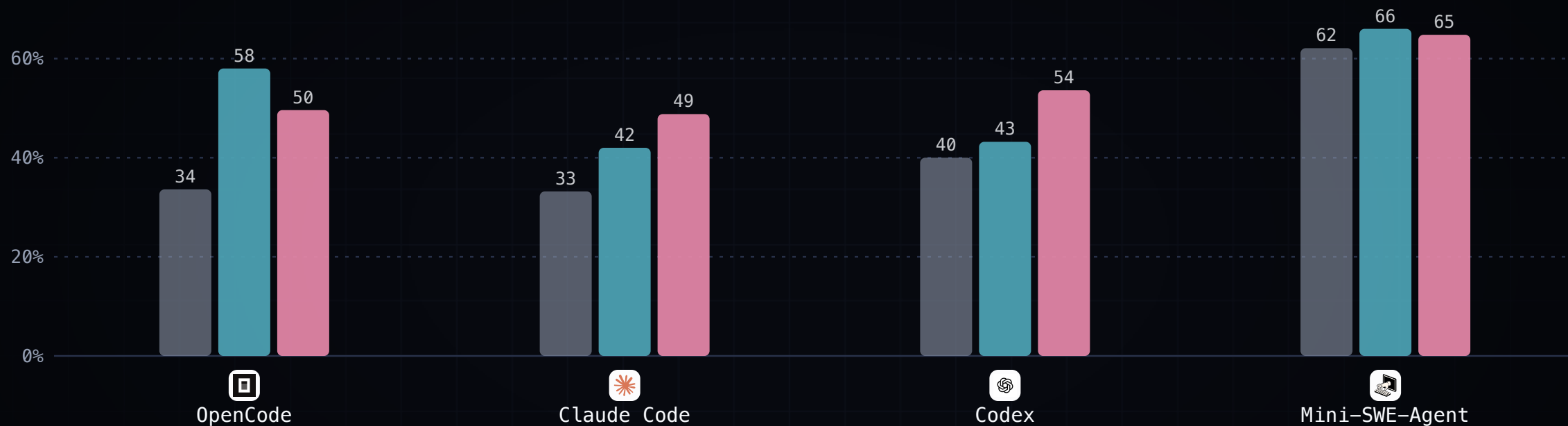
The bonus is tiny, and it is the reason the run works. GRPO learns from differences **inside a group**. When all eight rollouts are correct, correctness alone makes them identical and the group teaches nothing.

22.6% of multi-harness groups were all-correct with different tool counts.

For those, the bonus was the only thing telling the eight rollouts apart.



Where each run ends up, by harness



LEGEND

LFM base model LFM, OpenCode only LFM, multi-harness

Pass@1 at step 1,000 under each harness, next to the base model.



TRAINED ACROSS ALL FOUR

42% → 54%

held-out tasks solved, averaged over the four harnesses

31%

fewer tool calls

4 / 4

harnesses improved





Fewer calls, and fewer tokens with them

Tool calls

Generated tokens

Input tokens

Trained in OpenCode only

	OpenCode	Claude Code	Codex	Mini-SWE-Agent
100*	+26	-1	-3	+1
200*	+30	-10	+2	-2
300*	+31	+2	+7	-2
400*	+25	-2	+7	+2
500	+33	-5	+11	+1
600	+35	+9	+15	+4
700	+32	+7	+17	-1
800	+37	+6	+24	0
850				
900	+35	+7	+18	+3
1000	+36	-10	+24	+2

Trained across four harnesses

	OpenCode	Claude Code	Codex	Mini-SWE-Agent
100*	+23	+3	+15	+6
200*	+32	+10	+20	+7
300	+31	+12	+33	+7
400	+24	+19	+41	+13
500	+29	+20	+41	+13
600	+32	+16	+44	+12
700	+31	+19	+47	+15
800	+23	+18	+50	+11
850	+25	+16	+50	+16
900	+35	+18	+52	+15
1000	+33	+28	+53	+16

more than the base model fewer

Each cell compares a checkpoint with the base model on the test tasks both solved under that harness. Blue means fewer tool calls, orange means more. * marks an evaluation that was still missing a few cells.



What this does not prove

- ! Every run is a single seed. One model at 2.6B, 1,000 steps.
- ! The two runs did not see the same data: 162M tokens against 451M, because Claude Code rewrites its own history and each rollout became about eight training rows.
- ! The 1.9-point gap in overall accuracy is inside the noise. Read the per-harness split, because the average hides it.
- ! SFT and RL differ in objective, data source, task pool and compute, so that comparison is an observation, not a ranking.
- ! We still do not know why SFT hurt Mini-SWE-Agent.

Bigger runs on the same stack are going now.



The harness is half of it. The environments are the other half

The same labs that vary the harness also train over large pools of environments at once. Nothing in this stack is tied to data analysis, so that half is open too.

<u>MiMo-V2.6-RL</u>	7,780 of Xiaomi's own RL environments as Harbor tasks: code, cyber, terminal, webdev, music, general
<u>SmolDataEnvs</u>	5,000 data-analysis tasks, the ones behind every number in this talk
<u>Repo2RLEnv</u>	verifiable coding and terminal environments generated from real repositories
<u>Single-skill envs</u>	GeoGuesser, LaTeX-OCR, paint-with-code, Wordle, each one a Space you can open right now

You pick the harness and you pick the environments, and the trainer stays where it is.



Run it yourself

01	Install	openenv[harbor] on Python 3.12, and TRL from main until the worker ships
02	Pick a task set	any Harbor dataset on the Hub, ours or your own
03	Pick the harnesses	one string each, rotated per rollout
04	Run the example	examples/async_grpo_harbor, unchanged

[the example](#) [OpenEnv docs](#) [the tutorials](#)

A harness you did not write is now somewhere you can train.



Everything, in one place

READ

[The article](#)

[Why Harbor exists](#)

[RL environments guide](#)

[RL environments 101](#)

[FineEnvs Academy](#)

[The LFM2.5-2.6B release](#)

RUN

[Everything in one collection](#)

[TRL Harbor example](#)

[The tutorials](#)

[The environment, live](#)

[Harbor task visualiser](#)

[Every training curve](#)

MODELS AND DATA

[Multi-harness RL model](#)

[OpenCode-only RL model](#)

[The SFT models](#)

[The base model](#)

[SmolDataEnvs](#)

[MiMo-V2.6 RL environments](#)

DOCS AND CODE

[TRL docs](#)

[Async GRPO trainer](#)

[OpenEnv docs](#)

[Harbor](#)

[TRL on GitHub](#)

[OpenEnv on GitHub](#)

The deck is at hf.co/spaces/FineEnvs/multi-harness-rl-slides

Sergio Paniego [@sergiopaniego](#) Hugging Face



THESE SLIDES